

TUGAS ASSIGNMENT MINGGU 9
Komparasi Metode Segmentasi untuk Ekstraksi Objek
202523430040



OLEH:
Syifa Adilla Sukma
24343128

Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

TEKNIK INFORMATIKA
DEPARTEMEN ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026

BAB I

PENDAHULUAN

1. Tujuan Pratikum

- Memahami dan mengimplementasikan berbagai metode segmentasi citra.
- Membandingkan performa metode segmentasi berdasarkan metrik evaluasi.
- Menganalisis pengaruh karakteristik citra terhadap hasil segmentasi.

2. Dasar Teori

Segmentasi citra merupakan proses pemisahan objek dengan latar belakang pada citra digital. Segmentasi bertujuan untuk mempermudah analisis citra dengan membagi citra menjadi beberapa bagian atau region yang memiliki karakteristik tertentu.

Pada praktikum ini digunakan beberapa metode segmentasi citra, yaitu:

- Global Thresholding
- Otsu Thresholding
- Adaptive Mean Thresholding
- Adaptive Gaussian Thresholding
- Sobel
- Prewitt
- Canny
- Watershed
- Connected Component
- Region Growing

Evaluasi hasil segmentasi dilakukan menggunakan beberapa metrik berikut:

- IoU (Intersection over Union): Mengukur tingkat overlap antara hasil segmentasi dan ground truth.
- DICE Coefficient: Mengukur kesamaan area antara hasil segmentasi dan ground truth.
- Accuracy (ACC): Mengukur persentase piksel yang diprediksi dengan benar.
- Waktu Komputasi: Mengukur kecepatan metode dalam memproses citra.

BAB II

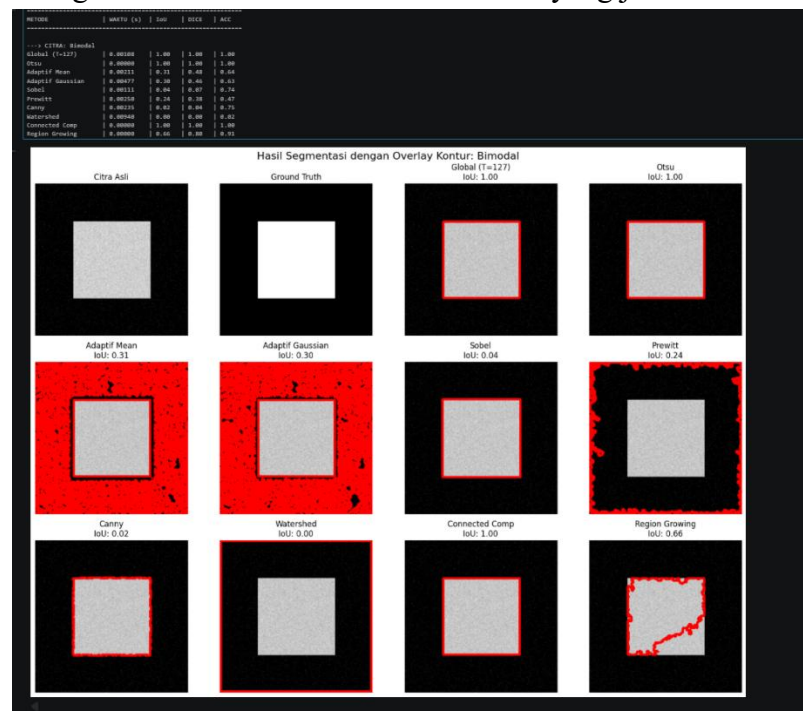
HASIL PERCOBAAN

1. Citra Bimodal

Berdasarkan hasil pengujian pada citra bimodal, diperoleh hasil evaluasi segmentasi sebagai berikut:

- Global Thresholding dan Otsu menghasilkan nilai IoU sebesar 1.00.
- Connected Component juga memberikan hasil sempurna dengan segmentasi objek yang sangat presisi.
- Adaptive Mean dan Adaptive Gaussian menghasilkan noise cukup besar sehingga nilai evaluasi menurun.
- Sobel, Prewitt, dan Canny hanya mendeteksi tepi objek.
- Watershed memberikan hasil segmentasi terburuk.

Hasil visualisasi segmentasi menunjukkan bahwa metode thresholding sangat cocok digunakan pada citra bimodal karena objek dan background memiliki distribusi intensitas yang jelas.



2. Citra Iluminasi Tidak Merata

Pada citra dengan pencahayaan tidak merata, hampir seluruh metode mengalami penurunan performa.

- Global Thresholding dan Otsu gagal memisahkan seluruh objek secara sempurna.
 - Adaptive Thresholding masih menghasilkan banyak area yang salah tersegmentasi.
 - Sobel dan Canny mampu mendeteksi bentuk lingkaran, tetapi tidak menghasilkan segmentasi area yang baik.
 - Connected Component hanya mampu mengambil sebagian area objek.
 - Region Growing gagal melakukan segmentasi secara optimal.
- Hal ini menunjukkan bahwa variasi pencahayaan menjadi tantangan utama bagi metode thresholding global.

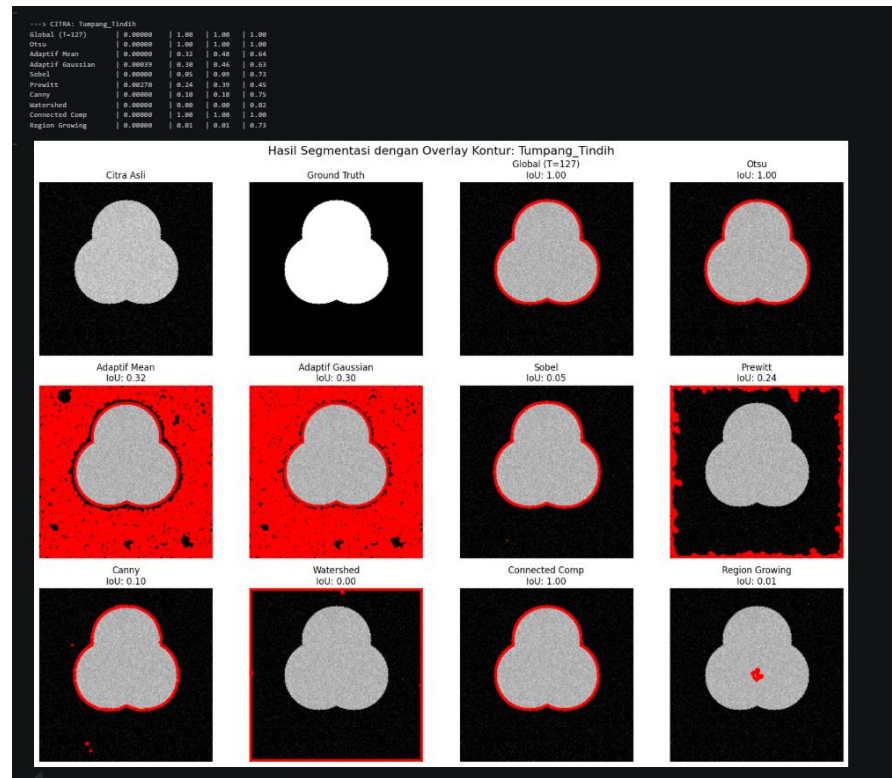


3. Citra Tumpang Tindih

Pada citra objek tumpang tindih diperoleh hasil sebagai berikut:

- Global Thresholding, Otsu, dan Connected Component kembali memberikan hasil terbaik dengan nilai evaluasi mendekati sempurna.
- Adaptive Mean dan Adaptive Gaussian menghasilkan banyak noise di background.
- Sobel dan Canny hanya menampilkan kontur objek.
- Region Growing gagal memisahkan area objek secara penuh.
- Watershed menghasilkan segmentasi yang tidak sesuai dengan ground truth.

Dari hasil tersebut dapat dilihat bahwa metode thresholding masih sangat efektif selama objek memiliki kontras yang jelas terhadap background.



BAB III

ANALISIS DAN PEMBAHASAN

Berdasarkan hasil percobaan yang telah dilakukan, dapat dianalisis beberapa hal berikut:

- **Metode Thresholding Memberikan Hasil Terbaik**
Metode Global Thresholding dan Otsu menunjukkan performa terbaik pada citra bimodal dan citra tumpang tindih. Hal ini disebabkan karena kedua citra memiliki distribusi intensitas yang jelas antara objek dan background.
- **Connected Component Sangat Efektif**
Metode Connected Component mampu mendeteksi area objek secara utuh sehingga menghasilkan nilai IoU dan DICE yang tinggi.
- **Pengaruh Iluminasi pada Segmentasi**
Pada citra dengan pencahayaan tidak merata, performa metode thresholding mengalami penurunan signifikan. Perubahan intensitas background menyebabkan objek sulit dipisahkan secara sempurna.
- **Metode Edge Detection Tidak Cocok untuk Segmentasi Area**
Metode Sobel, Prewitt, dan Canny hanya mendeteksi batas objek sehingga hasil evaluasi IoU dan DICE menjadi rendah ketika dibandingkan dengan ground truth berbasis area.
- **Watershed dan Region Growing Sensitif terhadap Parameter**
Metode Watershed dan Region Growing sangat bergantung pada marker dan seed awal. Kesalahan parameter menyebabkan over-segmentation maupun under-segmentation.

BAB IV

KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Pemilihan metode segmentasi sangat dipengaruhi oleh karakteristik citra.
2. Global Thresholding, Otsu, dan Connected Component memberikan performa terbaik pada citra bimodal dan citra tumpang tindih.
3. Variasi pencahayaan menyebabkan penurunan performa metode thresholding.
4. Metode edge detection lebih cocok digunakan untuk mendeteksi kontur objek dibandingkan segmentasi area.
5. Watershed dan Region Growing membutuhkan parameter yang tepat agar menghasilkan segmentasi optimal.

Link github

<https://github.com/Syifa0613/Tugas9>

Lampiran kode

```
1 # =====
2 # FUNGSI EVALUASI METRIK
3 # =====
4 def calculate_metrics(pred, gt):
5     pred = (pred > 0).astype(np.uint8)
6     gt = (gt > 0).astype(np.uint8)
7
8     TP = np.sum((pred == 1) & (gt == 1))
9     TN = np.sum((pred == 0) & (gt == 0))
10    FP = np.sum((pred == 1) & (gt == 0))
11    FN = np.sum((pred == 0) & (gt == 1))
12
13    iou = TP / (TP + FP + FN) if (TP + FP + FN) > 0 else 0
14    dice = (2 * TP) / ((2 * TP) + FP + FN) if ((2 * TP) + FP + FN) > 0 else 0
15    accuracy = (TP + TN) / (TP + TN + FP + FN)
16    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
17    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
18
19    return {"IoU": iou, "Dice": dice, "Acc": accuracy, "Prec": precision, "Rec": recall}
20
21 # =====
22 # 2. METODE AMBANG BATAS (THRESHOLDING)
23 # =====
24 def apply_thresholding(img):
25     results = {}
26
27     # Global Manual
28     start = time.time()
29     _, th_global = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
30     results['Global (T=127)'] = (th_global, time.time() - start)
31
32     # Otsu
33     start = time.time()
34     _, th_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
35     results['Otsu'] = (th_otsu, time.time() - start)
36
37     # Adaptive Mean & Gaussian
38     start = time.time()
39     th_adapt_mean = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)
40     results['Adaptif Mean'] = (th_adapt_mean, time.time() - start)
41
42     start = time.time()
43     th_adapt_gauss = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11,
44 2) results['Adaptif Gaussian'] = (th_adapt_gauss, time.time() - start)
45
46     return results
47
```



```

1 # =====
2 # 4. SEGMENTASI BERBASIS WILAYAH
3 # =====
4 def apply_region_based(img):
5     results = {}
6
7     # Watershed (Marker-based)
8     start = time.time()
9     _, thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
10    kernel = np.ones((3,3), np.uint8)
11    opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
12    sure_bg = cv2.dilate(opening, kernel, iterations=3)
13    dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
14    _, sure_fg = cv2.threshold(dist_transform, 0.5*dist_transform.max(), 255, 0)
15    sure_fg = np.uint8(sure_fg)
16    unknown = cv2.subtract(sure_bg, sure_fg)
17    _, markers = cv2.connectedComponents(sure_fg)
18    markers = markers + 1
19    markers[unknown == 255] = 0
20    img_color = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
21    cv2.watershed(img_color, markers)
22    ws_result = np.zeros_like(img)
23    ws_result[markers > 1] = 255 # Extract regions
24    results['Watershed'] = (ws_result, time.time() - start)
25
26    # Connected Components (Dari hasil Otsu)
27    start = time.time()
28    _, otsu_bin = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
29    num_labels, labels = cv2.connectedComponents(otsu_bin)
30    cc_result = (labels > 0).astype(np.uint8) * 255
31    results['Connected Comp'] = (cc_result, time.time() - start)
32
33    # Region Growing (FloodFill)
34    start = time.time()
35    h, w = img.shape
36    mask = np.zeros((h+2, w+2), np.uint8)
37    rg_result = img.copy()
38    # Asumsi seed point di tengah (untuk menyederhanakan)
39    seed_pt = (w//2, h//2)
40    cv2.floodFill(rg_result, mask, seed_pt, 255, 10, 10)
41    _, rg_bin = cv2.threshold(rg_result, 254, 255, cv2.THRESH_BINARY)
42    results['Region Growing'] = (rg_bin, time.time() - start)
43
44    return results
45

```

```

1 # =====
2 # 5. EKSEKUSI UTAMA & EVALUASI
3 # =====
4 images = create_synthetic_images()
5
6 print("="*60)
7 print(f"{'METODE':<20} | {'WAKTU (s)':<10} | {'IoU':<6} | {'DICE':<6} | {'ACC':<6}")
8 print("="*60)
9
10 for img_name, (img, gt) in images.items():
11     print(f"\n--> CITRA: {img_name}")
12
13     all_results = {}
14     all_results.update(apply_thresholding(img))
15     all_results.update(apply_edge_detection(img))
16     all_results.update(apply_region_based(img))
17
18     # Visualisasi Setup (3 Baris, 4 Kolom)
19     fig, axes = plt.subplots(3, 4, figsize=(16, 12))
20     fig.suptitle(f'Hasil Segmentasi dengan Overlay Kontur: {img_name}', fontsize=16)
21     axes = axes.ravel()
22
23     axes[0].imshow(img, cmap='gray')
24     axes[0].set_title('Citra Asli')
25     axes[0].axis('off')
26
27     axes[1].imshow(gt, cmap='gray')
28     axes[1].set_title('Ground Truth')
29     axes[1].axis('off')
30
31     idx = 2
32     for method_name, (pred_img, exec_time) in all_results.items():
33         metrics = calculate_metrics(pred_img, gt)
34         print(f"{'method_name':<20} | {'exec_time':<10.5f} | {'metrics['IoU']':<6.2f} | {'metrics['Dice']':<6.2f} | {'metrics['Acc']':<6.2f}")
35
36         if idx < len(axes):
37             # 1. Pastikan pred_img adalah gambar biner (0 atau 255) tipe uint8
38             pred_img_binary = (pred_img > 0).astype(np.uint8) * 255
39
40             # 2. Buat citra berwarna (RGB) dari citra asli sebagai background agar kontur terlihat merah
41             img_color_overlay = cv2.cvtColor(img.copy(), cv2.COLOR_GRAY2RGB)
42
43             # 3. Cari kontur dari hasil segmentasi prediksi
44             contours, _ = cv2.findContours(pred_img_binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
45
46             # 4. Gambar kontur pada citra background (warna merah, ketebalan 2 pixel agar jelas)
47             cv2.drawContours(img_color_overlay, contours, -1, (255, 0, 0), 2)
48
49             # Tampilkan hasilnya
50             axes[idx].imshow(img_color_overlay)
51             axes[idx].set_title(f"{'method_name':<20}\nIoU: {metrics['IoU']:.2f}")
52             axes[idx].axis('off')
53             idx += 1
54
55     # Sembunyikan subplot sisa (jika ada yang kosong di grid akhir)
56     for i in range(idx, len(axes)):
57         axes[i].axis('off')
58
59     plt.tight_layout()
60     plt.show()
61
62 print("\nProses Selesai. Gunakan data di atas untuk menyusun laporan Anda.")

```